EngageIQ (PACE) — Master Technical Specification & Architecture Document

The Single Source of Truth for Architecture, Algorithms, Data Pipelines, and Phased Implementation (Phases 2A – 4)

Table of Contents

- 1. Executive Summary & Document Authority
- 2. Project Philosophy & Core Framework
- 2.1 The Remote Work Visibility Gap & Anti-Surveillance Positioning
- 2.2 The Dual-Lens Model (Signal vs. Output)
- 2.3 Core Responsible AI Principles
- 3. Algorithmic & Mathematical Specifications
- 3.1 End-to-End Analytics Pipeline
- 3.2 Signal Layer: Engagement / Work-Pattern Health Index (EHS)
- 3.3 Output Layer: Output Productivity Index (OPI)
- 3.4 Role-Aware Normalization Matrix
- 3.5 PACE Fusion Model
- 3.6 Statistical Trend & Rolling Baseline Mechanics
- 3.7 Risk Band Classification & Escalation Engine
- 3.8 Explainable AI (XAI) Attribution Generation
- 4. System Architecture & Data Flows
- 4.1 High-Level Layered Architecture
- 4.2 Dual Ingestion Data Sequence
- 4.3 Target Technology Stack
- 5. Current Implementation Audit (Baseline Assessment)
- 5.1 Current State Summary
- 5.2 Implemented vs. Incomplete / Mocked Inventory
- 6. Complete Database Schema (DDL & SQLAlchemy 2.0 Models)
- 6.1 Entity Relationship Diagram
- 6.2 Detailed Model Definitions & Relationships
- 7. REST API Specification
- 7.1 Core Analytics Endpoints
- 7.2 Data Ingestion & Sync Endpoints
- 7.3 Integrations & Credential Vault Endpoints
- 7.4 Human Review & Governance Endpoints
- 8. Phase 2A Build Specification: Backend API Unification & Relational Migration
- 8.1 Objective & Architectural Scope
- 8.2 Database Migration & Seed Generator (`seed_data.py`)
- 8.3 FastAPI Controller Implementation
- 8.4 Frontend Client Rewiring (`client.ts`)

- 8.5 Phase 2A Verification & Test Suite
- 9. Phase 2B Build Specification: Integrations Management UI & Credential Vault
- 9.1 Objective & Architectural Scope
- 9.2 Frontend Integrations & Settings View (`/settings/integrations`)
- 9.3 Backend Fernet Credential Vault (`vault.py`)
- 9.4 Live Health-Check Ping Diagnostics
- 9.5 Dual-Mode Pipeline Controller (`sample` vs `live`)
- 9.6 Phase 2B Verification & Test Suite
- 10. Phase 3 Build Specification: Machine Learning Anomaly & Trend Engine
- 10.1 Objective: Elimination of Static Arithmetic
- 10.2 Data Preprocessing & Robust Scaling Pipeline
- 10.3 Scikit-Learn Isolation Forest Anomaly Detection
- 10.4 Deep Learning Time-Series Model: PyTorch LSTM Autoencoder
- 10.5 Burnout Pattern Heuristics & Escalation Logic
- 10.6 Explainable AI (XAI) Attribution Engine
- 10.7 Phase 3 Verification & Test Suite
- 11. Phase 4 Build Specification: Live SaaS Extractors & Sample-Data Elimination
- 11.1 Objective & Architectural Scope
- 11.2 GitHub REST/GraphQL Extractor
- 11.3 Jira Cloud REST Extractor
- 11.4 Slack Web API Metadata Extractor
- 11.5 Google Calendar API v3 Extractor
- 11.6 Background Ingestion Orchestrator & Sync Scheduler
- 11.7 Complete Elimination & Deprecation Plan for `sample_data.csv`
- 11.8 Production Hardening & Docker Containerization
- 12. Governance, Responsible AI & Privacy Guardrails
- 12.1 Surveillance Guardrails & Anti-Patterns
- 12.2 Human-in-the-Loop Protocol (The Seven Outcomes)
- 12.3 Data Minimization & Audit Logging

---

1. Executive Summary & Document Authority

This document is the **definitive Single Source of Truth (SSOT)** for EngageIQ (codenamed **PACE**). It provides the complete architectural, algorithmic, mathematical, database, and software engineering blueprint. It supersedes all prior preliminary notes.

EngageIQ is an enterprise-grade, privacy-conscious workplace analytics platform. It shifts organizations away from invasive surveillance (keystrokes, screen recordings, webcam monitoring, chat inspections) toward an objective, dual-lens evaluation of work metadata: 1. **Signal Layer (EHS - 40% weight):** Collaboration health, ergonomic pacing, meeting loads, and focus fragmentation. 2. **Output Layer (OPI - 60% weight):** Tangible task completion, code deliverables, deadline adherence, sprint velocity, and rework rates.

The platform fuses these layers into a unified **PACE score**, applies machine learning and statistical time-series anomaly detection to identify meaningful changes over time, and presents contextual risk signals for **Human-in-the-Loop** review.

↑ Back to Table of Contents

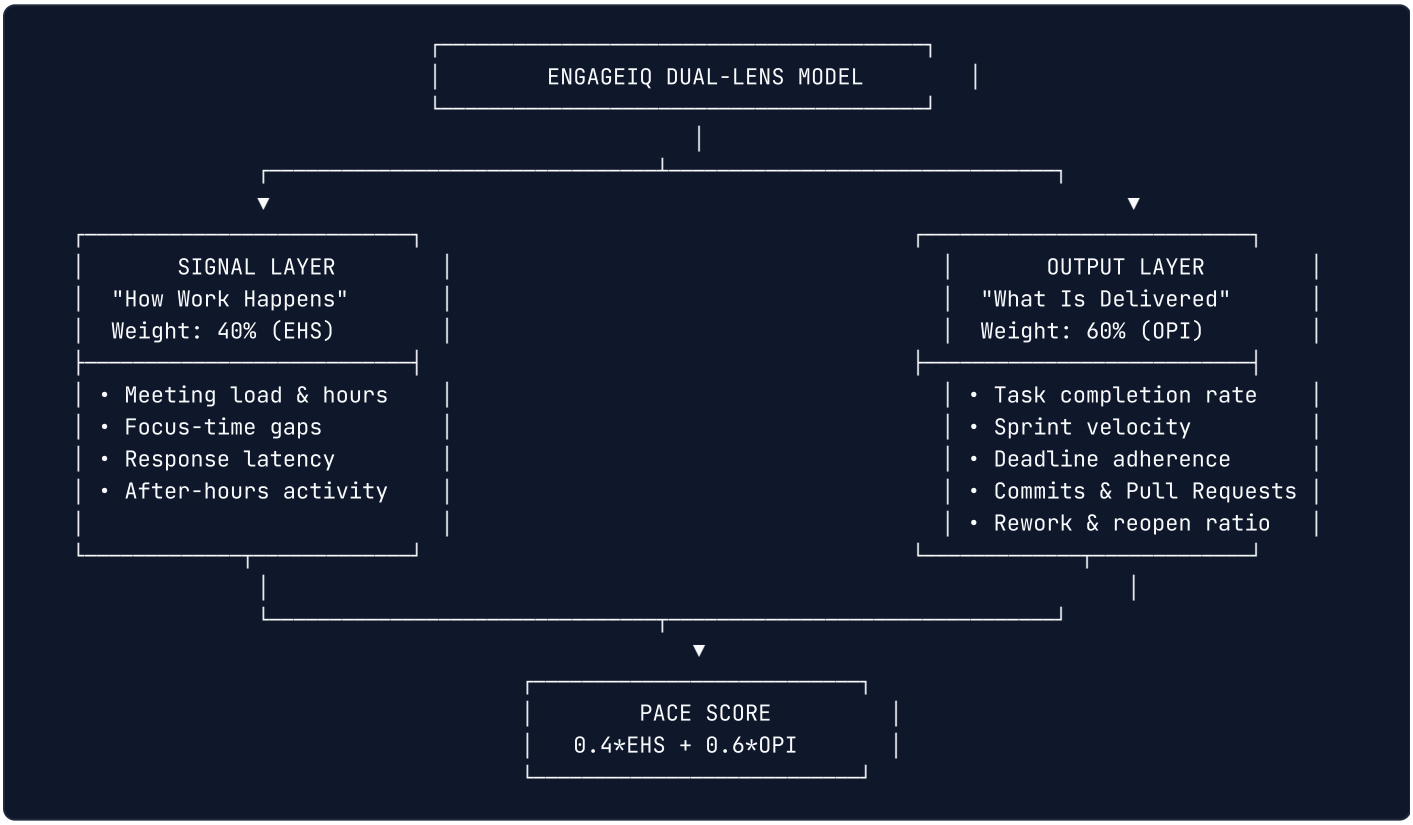
2. Project Philosophy & Core Framework

2.1 The Remote Work Visibility Gap & Anti-Surveillance Positioning

Remote and hybrid work eliminates informal physical observation. Traditional "bossware" attempts to fill this gap through invasive surveillance: - Keystroke logging, mouse tracking, and desktop video captures. - Sentiment analysis scanning employee chat content. - Continuous online/idle status tracking.

These surveillance mechanisms fail because they incentivize **"productivity theater"**, destroy psychological safety, trigger employee churn, and completely miss delivery blockers. EngageIQ operates under a strict anti-surveillance mandate: **analyze work patterns and deliverables from high-level metadata, never monitor personal behavior.**

2.2 The Dual-Lens Model (Signal vs. Output)



- **Output Layer is intentionally weighted higher (60%) than Signal Layer (40%):** This ensures introverted or autonomous employees with quieter communication habits are never penalized if their tangible delivery is strong.

2.3 Core Responsible AI Principles

1. **Privacy-First:** Strict data minimization. Only aggregated metadata is collected. Zero capture of email contents, chat message texts, screen pixels, or keystrokes.
2. **Human-in-the-Loop:** Analytics generate risk signals for human interpretation. The software never executes automated disciplinary actions, salary adjustments, or termination recommendations.
3. **Role Awareness:** Evaluation metrics are normalized against discipline baselines (Software Engineers vs. Sales Representatives vs. Designers).

4. **Trends Over Single-Point Snapshots:** Fluctuations within single weeks are normal. Alerts require multi-week persistent deviations or significant divergence from an individual's personal rolling baseline.

↑ Back to Table of Contents

---

3. Algorithmic & Mathematical Specifications

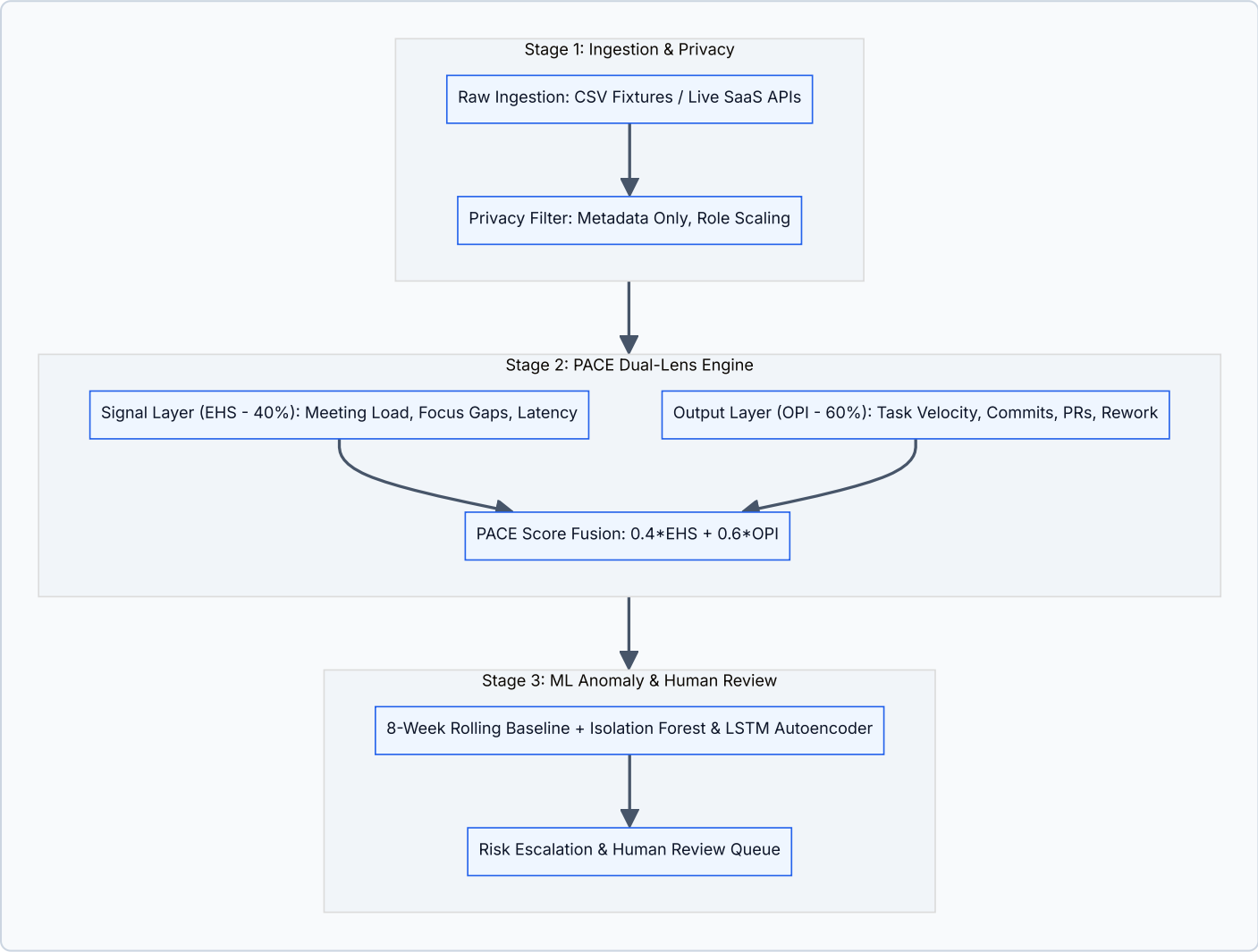
3.1 End-to-End Analytics Pipeline

The complete EngageIQ analytics journey spans from raw data ingestion through privacy-filtering, metric computation, machine learning trend analysis, and human review:



```
                                    |
                                    ▼
[PRESENTATION & GOVERNANCE]
┌──────────────────────────────────────────────────────────────────────┐
| • Manager Dashboard (Org Summary, Team Rosters, Employee Drill-Down)  |
| • Human-in-the-Loop Review Queue (Flagged employees + diagnostic evidence) |
| • Seven Approved Contextual Outcomes (No concern, workload rebalance, etc.) |
| • Immutable Audit Logging & Governance Trail                         |
└──────────────────────────────────────────────────────────────────────┘
```

Flowchart Specification (Mermaid)



3.2 Signal Layer: Engagement / Work-Pattern Health Index (EHS)

The **Engagement / Work-Pattern Health Index (EHS)** measures collaboration health and ergonomic pacing: - Range: $[0.0, 1.0]$, where $1.0 = \text{Optimal Health}$, $0.0 = \text{Severe Overwork / Fragmentation}$.

Mathematical Formulation

$$\text{EHS} = 1.0 - \left(W_1 \cdot \widetilde{m} + W_2 \cdot \widetilde{g} + W_3 \cdot \widetilde{l} + W_4 \cdot \widetilde{a} \right)$$

$$\text{EHS}_{\text{clipped}} = \max(0.0, \min(1.0, \text{EHS}))$$

Feature Weights & Definitions

Symbol	Feature Name	Description	Weight	Healthy Target
$\tilde{m}$	**Meeting Load Ratio**	Total weekly meeting hours divided by total available working hours (40 hrs).	$W_1 = 0.30$	$< 25\%$
$\tilde{g}$	**Focus Gap Ratio**	$1.0 - (\text{Longest uninterrupted } \geq 2\text{hr focus block in hours}/40)$.	$W_2 = 0.30$	Long blocks
$\tilde{l}$	**Response Latency**	Normalized median latency in responding to non-urgent direct communications.	$W_3 = 0.20$	Moderate pacing
$\tilde{a}$	**After-Hours Ratio**	Fraction of work activity occurring outside standard hours (7 PM – 8 AM or weekends).	$W_4 = 0.20$	$< 10\%$

3.3 Output Layer: Output Productivity Index (OPI)

The **Output Productivity Index (OPI)** measures tangible delivery and work throughput: - Range: $[0.0, 1.0]$, where 1.0 = \text{Maximum Throughput & Quality}, $0.0 = \text{Severe Bottleneck}$.

Mathematical Formulation

$$\text{OPI} = \left(W_1 \cdot \tilde{c} + W_2 \cdot \tilde{v} + W_3 \cdot \tilde{d} + W_4 \cdot \tilde{a}_{\text{code}}\right) - \left(W_5 \cdot \tilde{r}\right)$$

$$\text{OPI}_{\text{clipped}} = \max(0.0, \min(1.0, \text{OPI}))$$

Feature Weights & Definitions

Symbol	Feature Name	Description	Weight	Healthy Target
$\tilde{c}$	**Task Completion Rate**	Ratio of assigned tasks moved to `DONE` status vs. total assigned tasks.	$W_1 = 0.25$	$\geq 85\%$
$\tilde{v}$	**Sprint Velocity**	Delivered story points normalized against historical team expectations.	$W_2 = 0.25$	Consistent velocity
$\tilde{d}$	**Deadline Adherence**	Tasks completed on or before due date divided by total completed tasks.	$W_3 = 0.20$	$\geq 90\%$
$\tilde{a}_{\text{code}}$	**Deliverable Activity**	Volume of merged PRs, commits, or domain deliverables.	$W_4 = 0.20$	Steady output
$\tilde{r}$	**Rework / Reopen Ratio**	Tasks reopened due to bugs, QA rejections, or PR rework cycles.	$W_5 = 0.10$ *(Penalty)*	$< 10\%$

3.4 Role-Aware Normalization Matrix

Raw features are scaled against role expectations to prevent bias across disciplines:

$$\tilde{x} = \text{clip}\left(\frac{x_{\text{raw}}}{\text{Baseline}_{\text{role}}(x)}, 0.0, 1.0\right)$$

Role	Output Focus	Deliverable Target	Meeting Tolerance	Focus Block Priority
Software Engineer	PRs Merged, Commits, Story Points, Rework	3–6 PRs/wk, 15–30 pts/sprint	Low (< 10 hrs/wk)	Critical (≥ 4 hrs/day)
Product Designer	Figma Specs, Design Tickets, Revisions	4–8 tickets/sprint	Medium (< 15 hrs/wk)	High (≥ 3 hrs/day)
Sales Representative	CRM Pipeline, Client Calls, Deals Closed	25–40 touchpoints/wk	Very High ($20+$ hrs/wk)	Moderate
Engineering Manager	Team Unblocking, Sprint Planning, 1-on-1s	Team velocity predictability	High (18–25 hrs/wk)	Low

---

3.5 PACE Fusion Model

$$\mathbf{PACE} = (0.40 \times \text{EHS}) + (0.60 \times \text{OPI})$$

- If an employee has quiet collaboration habits ($\text{EHS} = 0.55$) but delivers exceptional tangible results ($\text{OPI} = 0.90$):

$$\text{PACE} = (0.40 \times 0.55) + (0.60 \times 0.90) = 0.22 + 0.54 = \mathbf{0.76} \quad (\text{Healthy})$$

- If an employee attends every meeting and responds instantly ($\text{EHS} = 0.95$) but delivers zero output ($\text{OPI} = 0.10$):

$$\text{PACE} = (0.40 \times 0.95) + (0.60 \times 0.10) = 0.38 + 0.06 = \mathbf{0.44} \quad (\text{Moderate / Blocked})$$

3.6 Statistical Trend & Rolling Baseline Mechanics

For employee e over rolling window $W = 8$ weeks:

$$\mu_e = \frac{1}{W-1}\sum_{i=1}^{W-1} \text{PACE}_{e,t-i}, \quad \sigma_e = \sqrt{\frac{1}{W-2}\sum_{i=1}^{W-1}(\text{PACE}_{e,t-i} - \mu_e)^2}$$

$$Z_t = \frac{\mu_e - \text{PACE}_{e,t}}{\sigma_e} \quad (\text{Positive } Z_t \text{ indicates deterioration})$$

Core Trend Signatures

1. **Sudden Change Anomaly:** $Z_t \geq 1.75$ and $Z_{t-1} < 1.0$.
2. **Persistent Deterioration:** $Z_t \geq 1.25$ and $Z_{t-1} \geq 1.25$ and $Z_{t-2} \geq 1.0$.
3. **Hidden Strain (Burnout Divergence):** $\Delta\text{EHS}_{\text{4-week}} \leq -0.15$ while $|\Delta\text{OPI}_{\text{4-week}}| \leq 0.05$.
4. **Dual Collapse:** $\Delta\text{EHS}_{\text{4-week}} \leq -0.10$ and $\Delta\text{OPI}_{\text{4-week}} \leq -0.10$.

3.7 Risk Band Classification & Escalation Engine

- **Healthy:** $\text{PACE} \geq 0.70$
- **Moderate:** $0.40 \leq \text{PACE} < 0.70$
- **At-Risk:** $\text{PACE} < 0.40$

Escalation Logic: If an employee has a score in the Healthy band (≥ 0.70) but exhibits **Persistent Deterioration** or **Hidden Strain**, their risk category is escalated:

$$\text{Healthy} \xrightarrow{\text{Persistent Anomaly}} \text{Moderate} \xrightarrow{\text{Severe Strain}} \text{At-Risk}$$

3.8 Explainable AI (XAI) Attribution Generation

Attribution calculations isolate which underlying metrics caused the deviation:

$$\text{Attribution}(f_j) = \frac{|x_{j,t} - \mu_{j,\text{baseline}}|}{\sum_k |x_{k,t} - \mu_{k,\text{baseline}}|}$$

Generated Narrative Example:

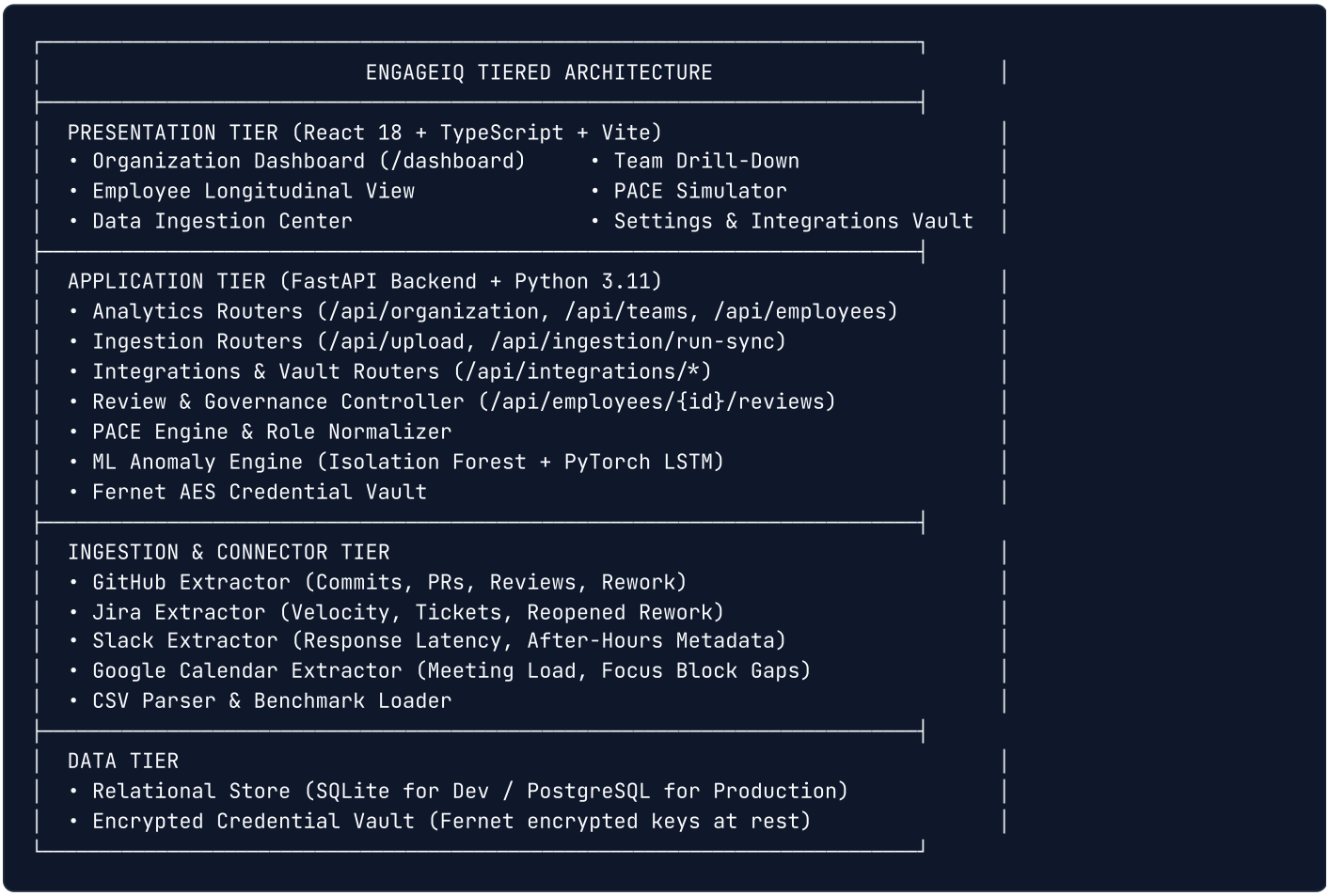
> *"Risk Band: Moderate (Escalated). Work-pattern health (EHS) fell 24% over 3 weeks due to meeting load increasing from 8.2 hrs to 19.5 hrs/week. Focus blocks declined by 65%. Delivery (OPI) remains steady at 0.82, signaling early burnout risk."*

↑ Back to Table of Contents

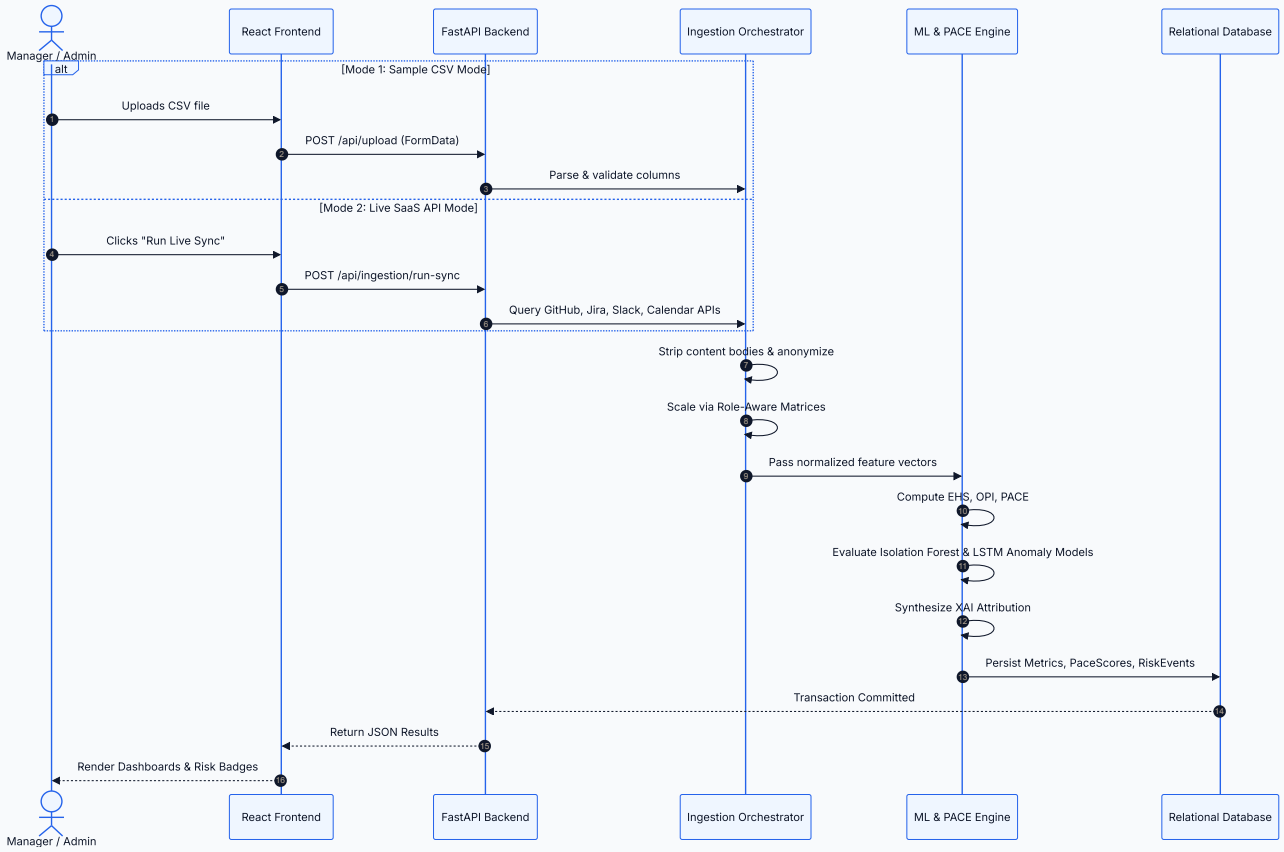
---

4. System Architecture & Data Flows

4.1 High-Level Layered Architecture



4.2 Dual Ingestion Data Sequence



4.3 Target Technology Stack

Component	Technology	Version	Key Responsibilities
Frontend Framework	React + TypeScript	18.x	Type-safe state, interactive components
Bundler & Tooling	Vite	6.x	Fast HMR, optimized production bundle
Styling	Vanilla CSS (CSS Variables)	CSS3	Fluid light/dark themes, zero-bloat glassmorphism
Backend Framework	FastAPI	0.115+	High-performance async REST API, auto OpenAPI docs
ASGI Server	Uvicorn	0.32+	Async Python HTTP server
ORM	SQLAlchemy	2.0+	Relational mapping, migrations, declarative models
Database	SQLite (Dev) / PostgreSQL (Prod)	3.x / 16.x	ACID compliant relational storage
Data Processing	Pandas + NumPy + SciPy	2.2+	Vectorized operations, rolling window calculations
Classical ML	Scikit-learn	1.5+	Isolation Forest anomaly detection, RobustScaler
Deep Learning	PyTorch	2.x	Time-series sequence autoencoder
Security & Cryptography	`cryptography` (Fernet)	43.x	Symmetric key encryption for stored API tokens

Component	Technology	Version	Key Responsibilities
HTTP Client	`httpx`	0.27+	Async HTTP client for external SaaS APIs

↑ Back to Table of Contents

5. Current Implementation Audit (Baseline Assessment)

5.1 Current State Summary

The project has a high-fidelity frontend UI and a minimal FastAPI backend. However, the system is currently in a prototype state where **the frontend dashboards consume static mock fixtures from `mock.ts`**, and **the backend only executes single-row linear arithmetic on uploaded CSV rows**.

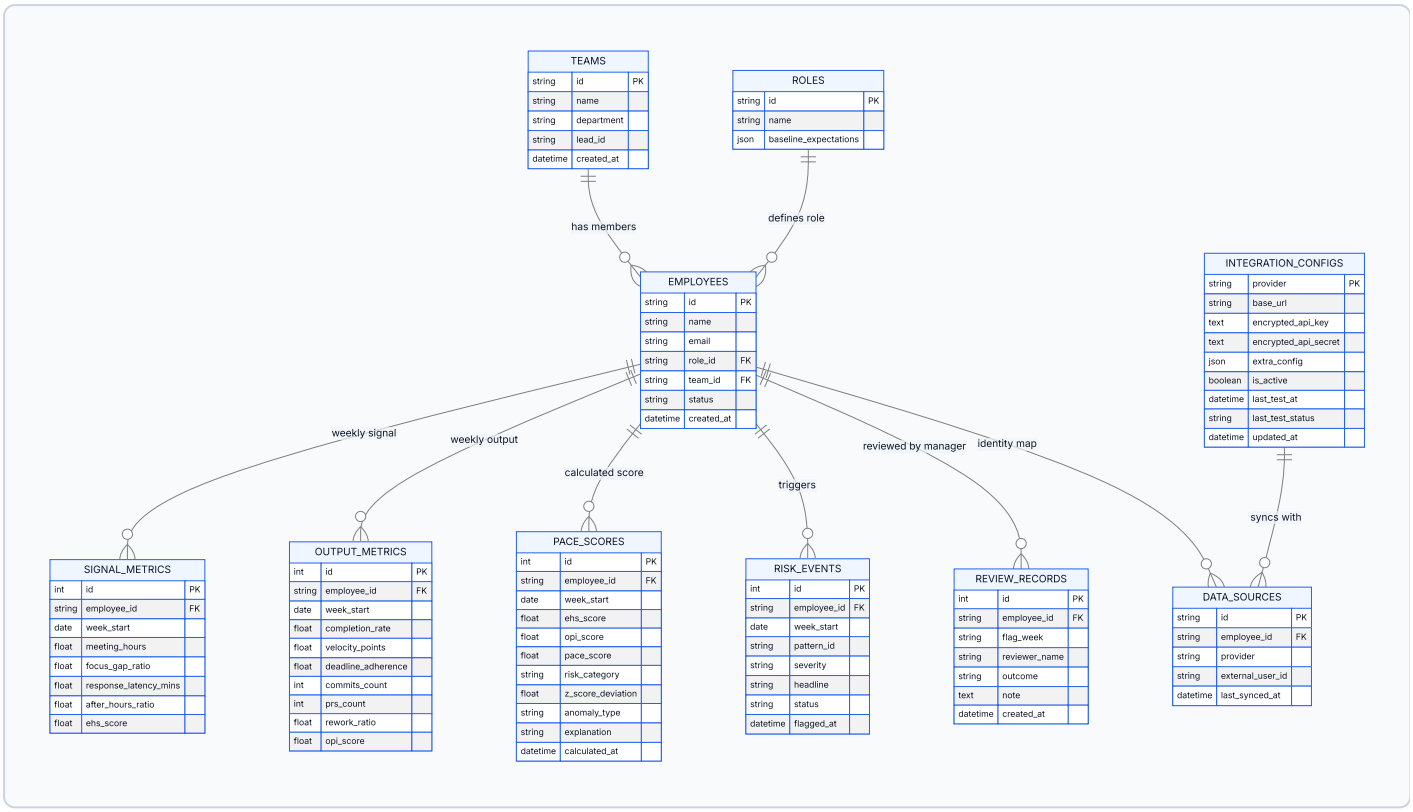
5.2 Implemented vs. Incomplete / Mocked Inventory

Subsystem	Implemented in Codebase	Incomplete / Mocked (Target for Next Phases)
Frontend Dashboards	Full UI views: Overview, OrgView, TeamView, EmployeeView, PaceScore simulator.	All views read static objects from `mock.ts`; not connected to live backend.
Data Ingestion UI	Drag-and-drop CSV uploader calling `POST /api/upload`.	Lacks Live Sync trigger, provider status indicators, and mode switcher.
Integrations UI	None.	Need dedicated `/settings/integrations` view with configuration modals.
Backend Models	Only basic `Employee` and `PaceScore` models in `models.py`.	Missing `Team`, `Role`, `SignalMetrics`, `OutputMetrics`, `RiskEvent`, `ReviewRecord`, `IntegrationConfig`.
REST Endpoints	Only `POST /api/upload` and `GET /api/scores` in `main.py`.	Missing `/api/organization`, `/api/teams/{id}`, `/api/employees/{id}`, `/api/integrations/*`.
Scoring Logic	Static linear formulas in `pace_engine.py` without baselines.	Needs rolling window baselines, role matrices, and machine learning models.
Anomaly Detection	Simple client-side heuristics in `src/lib/pace.ts`.	Needs backend Scikit-learn Isolation Forest and PyTorch LSTM engine.
Credential Security	None.	Needs Fernet symmetric encryption vault for external API keys.
External Connectors	None.	Needs GitHub, Jira, Slack, and Google Calendar extractors.

↑ Back to Table of Contents

6. Complete Database Schema (DDL & SQLAlchemy 2.0 Models)

6.1 Entity Relationship Diagram



6.2 Detailed Model Definitions & Relationships

```
# engageiq-backend/models.py (SQLAlchemy 2.0 Declarative Specification)
from datetime import datetime, date
from typing import List, Optional
from sqlalchemy import (
    Column, Integer, String, Float, DateTime, Date, ForeignKey, Boolean, Text, JSON
)
from sqlalchemy.orm import relationship, Mapped, mapped_column
from database import Base

class Team(Base):
    __tablename__ = "teams"
    id: Mapped[str] = mapped_column(String(64), primary_key=True)
    name: Mapped[str] = mapped_column(String(128), nullable=False)
    department: Mapped[str] = mapped_column(String(128), default="Engineering")
    lead_id: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    employees: Mapped[List["Employee"]] = relationship("Employee", back_populates="team")

class Role(Base):
    __tablename__ = "roles"
    id: Mapped[str] = mapped_column(String(32), primary_key=True)  # engineer, designer, salesperson, ma
    name: Mapped[str] = mapped_column(String(64), nullable=False)
    baseline_expectations: Mapped[dict] = mapped_column(JSON, default=dict)

    employees: Mapped[List["Employee"]] = relationship("Employee", back_populates="role")

class Employee(Base):
    __tablename__ = "employees"
    id: Mapped[str] = mapped_column(String(64), primary_key=True)
    name: Mapped[str] = mapped_column(String(128), index=True, nullable=False)
```

```
    email: Mapped[str] = mapped_column(String(128), unique=True, index=True, nullable=False)
    role_id: Mapped[str] = mapped_column(String(32), ForeignKey("roles.id"))
    team_id: Mapped[str] = mapped_column(String(64), ForeignKey("teams.id"))
    status: Mapped[str] = mapped_column(String(32), default="active")
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    team: Mapped["Team"] = relationship("Team", back_populates="employees")
    role: Mapped["Role"] = relationship("Role", back_populates="employees")
    signal_metrics: Mapped[List["SignalMetrics"]] = relationship("SignalMetrics", back_populates="employ
    output_metrics: Mapped[List["OutputMetrics"]] = relationship("OutputMetrics", back_populates="employ
    pace_scores: Mapped[List["PaceScore"]] = relationship("PaceScore", back_populates="employee")
    risk_events: Mapped[List["RiskEvent"]] = relationship("RiskEvent", back_populates="employee")
    reviews: Mapped[List["ReviewRecord"]] = relationship("ReviewRecord", back_populates="employee")
    data_sources: Mapped[List["DataSourceIdentity"]] = relationship("DataSourceIdentity", back_populates=

class DataSourceIdentity(Base):
    __tablename__ = "data_sources"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    employee_id: Mapped[str] = mapped_column(String(64), ForeignKey("employees.id"))
    provider: Mapped[str] = mapped_column(String(32))  # github, jira, slack, google_calendar
    external_user_id: Mapped[str] = mapped_column(String(128))  # username, email, or account_id
    last_synced_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

    employee: Mapped["Employee"] = relationship("Employee", back_populates="data_sources")

class SignalMetrics(Base):
    __tablename__ = "signal_metrics"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    employee_id: Mapped[str] = mapped_column(String(64), ForeignKey("employees.id"), index=True)
    week_start: Mapped[date] = mapped_column(Date, index=True, nullable=False)
    meeting_hours: Mapped[float] = mapped_column(Float, default=0.0)
    focus_gap_ratio: Mapped[float] = mapped_column(Float, default=0.0)
    response_latency_mins: Mapped[float] = mapped_column(Float, default=0.0)
    after_hours_ratio: Mapped[float] = mapped_column(Float, default=0.0)
    ehs_score: Mapped[float] = mapped_column(Float, default=1.0)

    employee: Mapped["Employee"] = relationship("Employee", back_populates="signal_metrics")

class OutputMetrics(Base):
    __tablename__ = "output_metrics"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    employee_id: Mapped[str] = mapped_column(String(64), ForeignKey("employees.id"), index=True)
    week_start: Mapped[date] = mapped_column(Date, index=True, nullable=False)
    completion_rate: Mapped[float] = mapped_column(Float, default=1.0)
    velocity_points: Mapped[float] = mapped_column(Float, default=0.0)
    deadline_adherence: Mapped[float] = mapped_column(Float, default=1.0)
    commits_count: Mapped[int] = mapped_column(Integer, default=0)
    prs_count: Mapped[int] = mapped_column(Integer, default=0)
    rework_ratio: Mapped[float] = mapped_column(Float, default=0.0)
    opi_score: Mapped[float] = mapped_column(Float, default=1.0)

    employee: Mapped["Employee"] = relationship("Employee", back_populates="output_metrics")

class PaceScore(Base):
    __tablename__ = "pace_scores"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    employee_id: Mapped[str] = mapped_column(String(64), ForeignKey("employees.id"), index=True)
    week_start: Mapped[date] = mapped_column(Date, index=True, nullable=False)
    ehs_score: Mapped[float] = mapped_column(Float, nullable=False)
    opi_score: Mapped[float] = mapped_column(Float, nullable=False)
    pace_score: Mapped[float] = mapped_column(Float, nullable=False)
    risk_category: Mapped[str] = mapped_column(String(32), default="healthy")  # healthy, moderate, at-r
    z_score_deviation: Mapped[float] = mapped_column(Float, default=0.0)
```

```
    anomaly_type: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
    explanation: Mapped[str] = mapped_column(Text, default="")
    calculated_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    employee: Mapped["Employee"] = relationship("Employee", back_populates="pace_scores")

class RiskEvent(Base):
    __tablename__ = "risk_events"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    employee_id: Mapped[str] = mapped_column(String(64), ForeignKey("employees.id"), index=True)
    week_start: Mapped[date] = mapped_column(Date, nullable=False)
    pattern_id: Mapped[str] = mapped_column(String(64), nullable=False)
    severity: Mapped[str] = mapped_column(String(32), default="medium")
    headline: Mapped[str] = mapped_column(String(256), nullable=False)
    status: Mapped[str] = mapped_column(String(32), default="needs-review")  # needs-review, reviewed
    flagged_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    employee: Mapped["Employee"] = relationship("Employee", back_populates="risk_events")

class ReviewRecord(Base):
    __tablename__ = "review_records"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    employee_id: Mapped[str] = mapped_column(String(64), ForeignKey("employees.id"), index=True)
    flag_week: Mapped[str] = mapped_column(String(32), nullable=False)
    reviewer_name: Mapped[str] = mapped_column(String(128), default="Engineering Lead")
    outcome: Mapped[str] = mapped_column(String(64), nullable=False)
    note: Mapped[str] = mapped_column(Text, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    employee: Mapped["Employee"] = relationship("Employee", back_populates="reviews")

class IntegrationConfig(Base):
    __tablename__ = "integration_configs"
    provider: Mapped[str] = mapped_column(String(32), primary_key=True)  # github, jira, slack, google_c
    base_url: Mapped[Optional[str]] = mapped_column(String(256), nullable=True)
    encrypted_api_key: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    encrypted_api_secret: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    extra_config: Mapped[dict] = mapped_column(JSON, default=dict)
    is_active: Mapped[bool] = mapped_column(Boolean, default=False)
    last_test_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
    last_test_status: Mapped[str] = mapped_column(String(32), default="untested")
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, onupdate=datetime.ut
```

↑ Back to Table of Contents

---

7. REST API Specification

7.1 Core Analytics Endpoints

`GET /api/organization`

Returns organizational headcount, aggregate weekly score history, current risk mix, team cards, and the review queue.

```
{
  "headcount": 12,
  "history": [
    { "week": "2026-08-03", "ehs": 0.74, "opi": 0.79, "pace": 0.77 },
```

```
      { "week": "2026-10-19", "ehs": 0.71, "opi": 0.76, "pace": 0.74 }
    ],
    "latest": { "week": "2026-10-19", "ehs": 0.71, "opi": 0.76, "pace": 0.74 },
    "delta": -0.03,
    "mix": { "healthy": 8, "moderate": 3, "at-risk": 1 },
    "teams": [
      {
        "team": { "id": "platform", "name": "Platform Core" },
        "headcount": 4,
        "latest": { "week": "2026-10-19", "ehs": 0.78, "opi": 0.81, "pace": 0.80 },
        "delta": 0.02,
        "mix": { "healthy": 3, "moderate": 1, "at-risk": 0 },
        "sparkline": [0.77, 0.78, 0.79, 0.80]
      }
    ],
    "queue": [
      {
        "employee": {
          "id": "emp-104",
          "name": "Diana Evans",
          "roleId": "engineer",
          "roleName": "Senior Engineer",
          "teamId": "platform",
          "teamName": "Platform Core"
        },
        "category": "at-risk",
        "pace": 0.38,
        "headline": "Collaboration health falling, delivery steady",
        "status": "needs-review"
      }
    ]
  }
```


`GET /api/teams/{team_id}`

Returns team metrics, member roster, sparklines, and team-specific queue items.

`GET /api/employees/{employee_id}`

Returns the employee's 12-week score trajectory, 4-week moving baseline, feature readings with percentage deltas, active risk assessments, and historical review notes.

7.2 Data Ingestion & Sync Endpoints

`POST /api/upload`

Accepts multipart CSV upload with required columns: `employee_id, employee_name, meeting_load, focus_gap, response_latency, after_hours_ratio, completion_rate, velocity, deadline_adherence, code_activity, rework_ratio`.

`POST /api/ingestion/run-sync`

Triggers an immediate background extraction from all active live connectors. - Request: `{"lookback_days": 7, "force_refresh": true}` - Response: `{"status": "queued", "sync_job_id": "sync_20261022_01", "active_providers": ["github", "jira"]}`

`GET /api/settings/mode`

Returns the active data pipeline mode (`"sample"` vs `"live"`).

`POST /api/settings/mode`

Switches pipeline execution mode. Rejects activation of `"live"` mode unless at least one Signal and one Output connector are tested and active.

7.3 Integrations & Credential Vault Endpoints

`GET /api/integrations/status`

Returns connection health, sync timestamps, and configured status for GitHub, Jira, Slack, and Google Calendar.

`POST /api/integrations/configure`

Encrypts and persists credentials for an external provider.

```
{
  "provider": "github",
  "base_url": "https://api.github.com",
  "api_key": "ghp_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
  "extra_config": {
    "org_name": "engageiq-org",
    "tracked_repos": ["core-engine", "web-dashboard"]
  }
}
```

`POST /api/integrations/test`

Performs an authenticated test ping against the external API and returns connection diagnostics.

7.4 Human Review & Governance Endpoints

`POST /api/employees/{employee_id}/reviews`

Saves a manager's contextual decision for a flagged risk event:

```
{
  "outcome": "discuss-workload",
  "note": "Diana was covering on-call rotation this sprint; meeting load spike is understood. Rebalancin
  "flagWeek": "2026-10-19"
}
```

↑ Back to Table of Contents

---

8. Phase 2A Build Specification: Backend API Unification & Relational Migration

8.1 Objective & Architectural Scope

Phase 2A unifies the frontend and backend. It eliminates `mock.ts` as the primary data provider by migrating SQLite to the full 10-table schema, building out all required FastAPI route handlers, and connecting `client.ts` to live endpoints.

8.2 Database Migration & Seed Generator (`seed_data.py`)

To ensure seamless testing and instant operational capability without manual CSV uploads: - Build `seed_data.py` to populate SQLite with the complete benchmark dataset: - 4 Teams: Platform Core, Growth & Marketing, Product Design, Mobile Apps. - 4 Roles: Software Engineer, Product Designer, Sales Representative, Engineering Manager. - 12 Employees with realistic historical trajectories. - 12 weeks of historical `SignalMetrics`, `OutputMetrics`, and `PaceScore` rows per employee ($12 \times 12 = 144$ data points). - Seed records illustrating specific test patterns: - *Alice Smith (Healthy benchmark)*: High steady delivery, balanced meetings. - *Bob Johnson (Moderate / Fragmented)*: Heavy meeting load, fragmented focus blocks. - *Diana Evans (At-Risk / Hidden Strain)*: Delivery steady (0.82) but EHS falling (0.28) due to $20+$ meeting hrs/wk. - Initial `RiskEvent` and `ReviewRecord` rows to populate the manager queue.

8.3 FastAPI Controller Implementation

Implement modular routers in `engageiq-backend/routers/`: 1. `routers/analytics.py`: - `get_organization_summary()`: Queries active employees, aggregates latest scores, computes rolling organization delta, groups members into team summaries, and formats pending review items. - `get_team_detail(team_id)`: Fetches team roster, team-level averages, sparklines, and member rows. - `get_employee_detail(employee_id)`: Assembles 12-week score histories, compares latest 4-week average to baseline, pulls granular feature readings, and fetches review records. 2. `routers/reviews.py`: - `create_employee_review(employee_id, review_input)`: Inserts `ReviewRecord`, resolves corresponding `RiskEvent.status = 'reviewed'`, and returns updated status.

8.4 Frontend Client Rewiring (`client.ts`)

Update `engageiq-frontend/src/api/client.ts`: - Ensure `VITE_API_URL` defaults to `http://localhost:8000`. - Remove fallback to `mock.ts` when backend is available. - Implement robust HTTP error handling with status code mapping: - `404`: Record not found message. - `500`: Backend computation exception with user-friendly retry toast. - Timeout handling after 8 seconds.

8.5 Phase 2A Verification & Test Suite

1. Run backend migration and seed script: `python seed_data.py`.
2. Start FastAPI server (`uvicorn main:app --reload`) and Vite dev server (`npm run dev`).
3. Verify that the Organization Dashboard, Team View, and Employee Drill-Down load data from SQLite.
4. Submit a review on an At-Risk employee; verify the review note is stored in SQLite and the review badge updates dynamically.

↑ Back to Table of Contents

---

9. Phase 2B Build Specification: Integrations Management UI & Credential Vault

9.1 Objective & Architectural Scope

Phase 2B gives managers full visibility and control over external data connections via a dedicated **Integrations & Settings** view. It implements an AES encryption vault for API tokens, provides live connection diagnostics, and adds a runtime switch between Sample Data and Live SaaS modes.

9.2 Frontend Integrations & Settings View (`/settings/integrations`)

Navigation Integration

Add **"Settings"** to the primary header in `Layout.tsx` and register route `/settings/integrations` in `App.tsx`.

UI Component Hierarchy

```
src/pages/IntegrationsSettings.tsx
├── PipelineModeSwitcher.tsx        // Toggle: [●] Sample Data  [ ] Live SaaS APIs
├── ProviderCardGrid.tsx
│   ├── GitHubIntegrationCard.tsx   // Commits, PRs, Reviews
│   ├── JiraIntegrationCard.tsx     // Sprints, Velocity, Issues
│   ├── SlackIntegrationCard.tsx    // Response Latency, After-Hours
│   └── CalendarIntegrationCard.tsx // Meeting Hours, Focus Blocks
├── CredentialConfigModal.tsx       // Glassmorphism modal for token/URL entry
└── ConnectionTestBadge.tsx         // Live status chip with error tooltip
```

Modal Interaction & Fields

- **GitHub:** Endpoint URL (default: `https://api.github.com`), Personal Access Token (PAT) / GitHub App Token, Tracked Organization / Repository list.
- **Jira:** Domain URL (`https://company.atlassian.net`), User Email, API Token, Tracked Project Keys.
- **Slack:** Bot User OAuth Token (`xoxb-...`), Workspace ID, Channel Inclusions.
- **Google Calendar:** Service Account JSON / OAuth Client ID & Secret, Domain delegation.
- **Actions:**
- **"Test Connection"**: Triggers `POST /api/integrations/test` with real-time spinner and result badge.
- **"Save & Encrypt"**: Triggers `POST /api/integrations/configure` and closes modal.

9.3 Backend Fernet Credential Vault (`vault.py`)

```
# engageiq-backend/vault.py
import os
import base64
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC

MASTER_KEY_ENV = os.getenv("ENGAGEIQ_MASTER_VAULT_KEY", "default-engageiq-dev-secret-key-32b")

def _get_fernet_instance() -> Fernet:
    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=32,
        salt=b"engageiq_vault_salt_constant",
        iterations=100000,
    )
    derived_key = base64.urlsafe_b64encode(kdf.derive(MASTER_KEY_ENV.encode()))
    return Fernet(derived_key)

def encrypt_credential(plain_text: str) -> str:
    if not plain_text:
        return ""
    fernet = _get_fernet_instance()
    return fernet.encrypt(plain_text.encode("utf-8")).decode("utf-8")

def decrypt_credential(cipher_text: str) -> str:
    if not cipher_text:
        return ""
    fernet = _get_fernet_instance()
    return fernet.decrypt(cipher_text.encode("utf-8")).decode("utf-8")

def mask_token(token: str) -> str:
    if not token or len(token) < 8:
```

```
        return "••••••••"
    return f"{token[:4]}••••••••{token[-4:]}"
```

9.4 Live Health-Check Ping Diagnostics

In `routers/integrations.py`, the test endpoint executes a lightweight authenticated request against the provider's API: - **GitHub:** `GET https://api.github.com/user` with `Authorization: Bearer <token>`. Checks rate-limit headers (`X-RateLimit-Remaining`). - **Jira:** `GET https://{domain}/rest/api/3/myself` with HTTP Basic Auth (`email:token`). - **Slack:** `POST https://slack.com/api/auth.test` with `Authorization: Bearer <token>`. - **Google Calendar:** `GET https://www.googleapis.com/calendar/v3/users/me/calendarList?maxResults=1`.

9.5 Dual-Mode Pipeline Controller (`sample` vs `live`)

- Endpoint `POST /api/settings/mode` manages the global execution state.
- **Switching Rule:** The system disallows switching to `"live"` mode unless:
- At least one Signal Layer provider (Google Calendar or Slack) is connected and tested.
- At least one Output Layer provider (GitHub or Jira) is connected and tested.
- If conditions are met, the ingestion engine pulls data via live extractors; otherwise, it operates in `"sample"` mode using benchmark SQLite records and CSV fixtures.

9.6 Phase 2B Verification & Test Suite

1. Navigate to `/settings/integrations`.
2. Open the GitHub configuration modal, enter a sample personal access token, and click "Test Connection".
3. Verify that the backend securely encrypts the token in `integration_configs`.
4. Verify that the UI displays masked tokens (`ghp_••••••••3a9f`) and that status chips render connection health accurately.

↑ Back to Table of Contents

---

10. Phase 3 Build Specification: Machine Learning Anomaly & Trend Engine

10.1 Objective: Elimination of Static Arithmetic

Phase 3 replaces the basic single-row equations in `pace_engine.py` with a true statistical and machine learning pipeline that models sequential work patterns, identifies multi-metric strain signatures, and outputs explainable attributions.

10.2 Data Preprocessing & Robust Scaling Pipeline

For each employee i at week t, metrics are organized into a 9-dimensional vector:

$$\mathbf{x}_{i,t} = \left[\widetilde{m}_{i,t} \quad \widetilde{g}_{i,t} \quad \widetilde{l}_{i,t} \quad \widetilde{a}_{i,t} \quad \widetilde{c}_{i,t} \quad \widetilde{v}_{i,t} \quad \widetilde{d}_{i,t} \quad \widetilde{a}_{\text{code},i,t} \quad \widetilde{r}_{i,t}\right]^T$$

- **Normalization:** Scikit-Learn's `RobustScaler(quantile_range=(15.0, 85.0))` is applied per role group to ensure scaling is resilient against extreme outliers.

10.3 Scikit-Learn Isolation Forest Anomaly Detection

To capture non-linear, cross-layer anomalies (e.g., normal meeting hours paired with abnormal burst activity at midnight): - **Model:** `sklearn.ensemble.IsolationForest` - **Parameters:** - `n_estimators = 150` - `contamination = 0.08` (expected anomaly rate across typical organization) - `max_samples = 'auto'` - `random_state = 42` -

Scoring: The model outputs an anomaly score $s \in [-1, 1]$. Observations with $s < 0$ are marked as potential structural work-pattern anomalies and submitted to the attribution engine.

10.4 Deep Learning Time-Series Model: PyTorch LSTM Autoencoder

For organizations with ≥ 8 weeks of historical records, EngageIQ deploys a lightweight **LSTM Autoencoder** to detect temporal sequence degradation:

```
# engageiq-backend/ml/lstm_model.py
import torch
import torch.nn as nn

class TemporalLSTMAutoencoder(nn.Module):
    def __init__(self, seq_len=4, n_features=9, hidden_dim=16):
        super().__init__()
        self.seq_len = seq_len
        self.n_features = n_features

        # Encoder
        self.encoder = nn.LSTM(
            input_size=n_features,
            hidden_size=hidden_dim,
            num_layers=1,
            batch_first=True
        )
        # Latent Bottleneck
        self.bottleneck = nn.Linear(hidden_dim, hidden_dim // 2)
        self.unbottleneck = nn.Linear(hidden_dim // 2, hidden_dim)

        # Decoder
        self.decoder = nn.LSTM(
            input_size=hidden_dim,
            hidden_size=n_features,
            num_layers=1,
            batch_first=True
        )

    def forward(self, x):
        # x shape: [batch_size, seq_len=4, n_features=9]
        _, (h_n, _) = self.encoder(x)
        latent = torch.relu(self.bottleneck(h_n.squeeze(0)))
        expanded = torch.relu(self.unbottleneck(latent)).unsqueeze(1).repeat(1, self.seq_len, 1)
        out, _ = self.decoder(expanded)
        return out  # Reconstructed sequence: [batch_size, seq_len, n_features]

def compute_reconstruction_error(model: TemporalLSTMAutoencoder, sequence_tensor: torch.Tensor) → float
    model.eval()
    with torch.no_grad():
        reconstructed = model(sequence_tensor)
        loss = nn.functional.mse_loss(reconstructed, sequence_tensor, reduction="mean")
    return float(loss.item())
```

- **Inference Logic:** If sequence reconstruction error exceeds threshold $\tau = \mu_{\text{train_error}} + 2.5\sigma_{\text{train_error}}$, a temporal trajectory anomaly is confirmed.

10.5 Burnout Pattern Heuristics & Escalation Logic

The ML engine checks for four explicit behavioral patterns:

```
def classify_trend_pattern(history_scores: List[dict]) → Optional[str]:
    # Requires last 4 weeks of scores
    if len(history_scores) < 4:
        return None
    recent = history_scores[-4:]
    first, last = recent[0], recent[-1]

    ehs_drop = first['ehs'] - last['ehs']
    opi_drop = first['opi'] - last['opi']

    # 1. Hidden Strain: EHS dropping sharply while OPI stays steady
    if ehs_drop ⩾ 0.12 and abs(opi_drop) ⩽ 0.05:
        return "collab-down-output-flat"

    # 2. Dual Collapse: Both EHS and OPI dropping
    if ehs_drop ⩾ 0.10 and opi_drop ⩾ 0.10:
        return "both-layers-down"

    # 3. Persistent Low Health: PACE under 0.70 for 3+ consecutive weeks
    if all(w['pace'] < 0.70 for w in recent[-3:]) and (first['pace'] - last['pace']) ⩾ 0.05:
        return "persistent-decline"

    # 4. Sudden Divergence: Latest PACE > 0.15 below historical average
    baseline_pace = sum(w['pace'] for w in history_scores[:-1]) / (len(history_scores) - 1)
    if (baseline_pace - last['pace']) ⩾ 0.15:
        return "sudden-change"

    return None
```

10.6 Explainable AI (XAI) Attribution Engine

The engine computes relative feature contributions and formats human-readable narratives:

```
def generate_xai_explanation(current_features: dict, baseline_features: dict, pattern: Optional[str]) →
    deltas = {}
    for key in current_features:
        base = baseline_features.get(key, 0.0)
        curr = current_features[key]
        if base > 0:
            deltas[key] = (curr - base) / base

    # Find the top deteriorating factors
    sorted_factors = sorted(deltas.items(), key=lambda x: x[1], reverse=True)

    narratives = []
    if pattern == "collab-down-output-flat":
        narratives.append("Work-pattern health (EHS) falling while delivery remains steady (early strain
    elif pattern == "both-layers-down":
        narratives.append("Both collaboration health and tangible delivery show concurrent declines.")

    for metric, pct in sorted_factors[:2]:
        if pct > 0.15:
            readable = metric.replace('_', ' ').title()
            narratives.append(f"{readable} increased by {int(pct*100)}% over historical baseline.")

    return " ".join(narratives) if narratives else "Scores within normal operating variance."
```

10.7 Phase 3 Verification & Test Suite

1. Run automated unit tests comparing baseline statistical calculations against scikit-learn outputs.
2. Ingest synthetic employee data exhibiting Hidden Strain; confirm that the ML engine flags the pattern, escalates the risk band from Healthy to Moderate, and attaches the attribution narrative.

↑ Back to Table of Contents

---

11. Phase 4 Build Specification: Live SaaS Extractors & Sample-Data Elimination

11.1 Objective & Architectural Scope

Phase 4 implements live, modular SaaS connectors for GitHub, Jira, Slack, and Google Calendar. It establishes scheduled incremental synchronization and removes operational dependency on `sample_data.csv`.

11.2 GitHub REST/GraphQL Extractor

- **Target File:** `engageiq-backend/connectors/github_connector.py`
- **Extracted Metrics:**
- Total weekly commits authored per employee.
- Pull Requests opened and merged.
- Review turnaround latency (hours from PR assigned to review submitted).
- Rework commits (subsequent commits on the same branch following review comments).
- **Privacy Enforcement:** Strips all code diffs, file paths, and commit messages. Only processes timestamps, counts, author identifiers, and status strings.
- **Rate-Limit Resilience:** Monitors `X-RateLimit-Remaining`. Automatically halts and sleeps if quota falls below 50 calls.

11.3 Jira Cloud REST Extractor

- **Target File:** `engageiq-backend/connectors/jira_connector.py`
- **Extracted Metrics:**
- Tasks moved to `DONE` status during the weekly cycle.
- Total story points resolved vs. planned sprint commitments.
- Deadline adherence (resolution date $\leq$ due date).
- Reopened task occurrences (status transition from `Done` or `In Review` back to `In Progress`).
- **Privacy Enforcement:** Ignores issue summaries, ticket descriptions, and user comments. Only extracts issue type, point values, timestamps, and status history.

11.4 Slack Web API Metadata Extractor

- **Target File:** `engageiq-backend/connectors/slack_connector.py`
- **Extracted Metrics:**
- Volume of messages sent during business hours vs. after-hours ($7\ \text{PM} - 8\ \text{AM}$).
- Median response latency to direct tags and mentions during working hours.
- **Strict Privacy Ban: Zero message content reading.** The API request queries metadata fields only (`user`, `ts`, `thread_ts`, `reply_count`). Text strings are explicitly ignored and never parsed or stored.

11.5 Google Calendar API v3 Extractor

- **Target File:** `engageiq-backend/connectors/calendar_connector.py`
- **Extracted Metrics:**
- Total weekly meeting duration in hours.
- Meeting fragmentation: Calculates focus-time blocks (uninterrupted intervals ≥ 2 hours between 9 AM and 5 PM).
- After-hours meetings (scheduled outside core working hours).
- **Privacy Enforcement:** Ignores meeting titles, attendee identities, and calendar notes. Reads only `start.dateTime` and `end.dateTime`.

11.6 Background Ingestion Orchestrator & Sync Scheduler

- **Target File:** `engageiq-backend/sync_orchestrator.py`
- **Job Flow:**
- Reads active connectors from `integration_configs`.
- Decrypts credentials in-memory via `vault.py`.
- Executes extractor fetch calls concurrently using `asyncio` and `httpx.AsyncClient`.
- Maps external user IDs (GitHub handles, Slack IDs, Jira emails) to internal `employees.id` via `data_sources` table.
- Feeds aggregated weekly metrics to the ML scoring engine.
- Commits new `SignalMetrics`, `OutputMetrics`, and `PaceScore` rows to the database.
- **Scheduling:** Runs automatically every Monday at 00:00 UTC via FastAPI background tasks or an external cron runner.

11.7 Complete Elimination & Deprecation Plan for `sample_data.csv`

1. **Deprecation Phase:** When `DataSourceMode.LIVE` is activated, the CSV ingestion endpoint is disabled in the UI.
2. **Fallback Isolation:** `sample_data.csv` is relocated to a dedicated benchmark fixtures directory (`tests/fixtures/benchmark_data.csv`) and used exclusively for automated integration tests and offline demonstrations.
3. **Operational State:** The production platform operates entirely on live SaaS extracts with zero dependency on static files.

11.8 Production Hardening & Docker Containerization

- **PostgreSQL Configuration:** Add production database connection pooling via asyncpg.
- **Containerization:**
- `backend.Dockerfile`: Multi-stage build running FastAPI via Uvicorn under an unprivileged user.
- `frontend.Dockerfile`: Multi-stage build compiling React/TypeScript and serving via high-performance Nginx.
- `docker-compose.yml`: Coordinates PostgreSQL, FastAPI backend, and Nginx frontend with health checks.

↑ Back to Table of Contents

---

12. Governance, Responsible AI & Privacy Guardrails

12.1 Surveillance Guardrails & Anti-Patterns

EngageIQ is architecturally engineered to make invasive surveillance impossible:

Prohibited Invasive Method	EngageIQ Architectural Equivalent
Keystroke logging / typing speed counters	Tangible task completion in project trackers (Jira/GitHub)
Screen recording / periodic desktop captures	Verified code reviews and merged deliverables
Webcam monitoring / gaze tracking	**Banned.** Zero capture capability in codebase
Reading private chat messages or email text	Interaction timestamps and response latency metadata only
Continuous online / idle status tracking	Sprint-level deliverable outcomes over time

12.2 Human-in-the-Loop Protocol (The Seven Outcomes)

Automated algorithms never take disciplinary action. When an employee is flagged in the review queue, the manager must evaluate the evidence and document one of seven structured outcomes: 1. **No concern:** Clear, harmless context (e.g., approved PTO, planned conference). 2. **Monitor trend:** Pattern is minor; continue observing over next 2 sprints. 3. **Discuss workload:** Meeting overload or sprint allocation requires rebalancing. 4. **Check project blockers:** Investigating external dependencies or access issues. 5. **Offer support:** Proactive outreach for wellness check-in or fatigue mitigation. 6. **Correct a data issue:** A connector identity mapping was erroneous. 7. **Override the risk signal:** Dismiss the automated alert with a required explanatory rationale.

12.3 Data Minimization & Audit Logging

- **Audit Trails:** All manager reviews, overrides, and connector credential modifications are committed to `audit_logs` with timestamps, reviewer user IDs, and outcome rationale.
- **Data Pruning:** Granular event metadata is pruned after 12 months rolling history; only anonymized rolling score baselines are retained for longitudinal trend modeling.

↑ Back to Table of Contents

End of Specification. This document is the authoritative Single Source of Truth for the EngageIQ platform.